Week 1 — Planning, setup, and database

Both partners

- Confirm the pair number, cities, seed, and port.
- Create the private Git repository.
- Decide the database schema and API structure.
- Set up the project folders.
- Configure MySQL, FastAPI, and React.
- Create the README and project setup instructions.
- Establish Git branches and commit rules.

You

- Set up the FastAPI backend.
  Implemented: FastAPI route modules cover authentication, student profiles, jobs, applications, events, and assistant features.
  How it works and why it matters: backend/app/main.py starts the service and exposes API documentation so the React frontend has one consistent service.
  Supporting files: backend/app/main.py; backend/app/api/routes/; backend/app/api/dependencies.py. Suggested evidence: sanitized Swagger UI route list.
- Create database models and connections.
  Implemented: the backend connects to MySQL database p16_handshake and uses SQLAlchemy models and sessions.
  How it works and why it matters: backend/app/core/config.py and backend/app/core/database.py manage the connection, while models represent students, jobs, applications, events, and registrations.
  Supporting files: backend/app/core/config.py; backend/app/core/database.py; backend/app/models/; database/01_create_database.sql; database/02_schema.sql. Suggested evidence: read-only table verification.
- Implement password hashing.
  Implemented: passwords are protected with bcrypt before storage.
  How it works and why it matters: the security helpers hash passwords during signup and verify them during login without storing plain-text passwords.
  Supporting files: backend/app/core/security.py; backend/app/api/routes/auth.py; backend/tests/test_security.py. Suggested evidence: sanitized hashing and verification test results.
- Start student authentication and profile APIs.
  Implemented: student signup, login, logout state clearing, JWT authorization, and protected student profile APIs.
  How it works and why it matters: protected routes identify the current student from the JWT and role checks prevent a student identity from being used as a company identity.

Supporting files: backend/app/api/routes/auth.py; backend/app/api/dependencies.py; backend/app/api/routes/student_profile.py; frontend/src/services/authService.js; backend/tests/test_jwt.py. Suggested evidence: redacted signup, login, protected request, and logout flow.

Your partner

- Set up the React frontend.
- Create the navigation, login, signup, and dashboard pages.
- Start company authentication and profile APIs.
- Create the initial Bootstrap layout.

End-of-week goal

The frontend, backend, and database should communicate successfully. Students and companies should be able to register and sign in.

Week 2 — Core recruiting platform

You

Build the student workflow:

- Student profile page
  Implemented: the student profile flow lets an authenticated student view and update profile information.
  How it works and why it matters: frontend profile requests go through the student service, while the protected API validates and stores the matching student's data.
  Supporting files: frontend/src/pages/StudentDashboard.jsx; frontend/src/pages/StudentProfileView.jsx; frontend/src/services/studentService.js; backend/app/api/routes/student_profile.py. Suggested evidence: redacted profile page and save result.
- Education, skills, experience, and contact information
  Implemented: the profile supports education, skills, experience, and contact fields.
  How it works and why it matters: the React form sends these fields to the student profile API, which validates and associates them with the authenticated student.
  Supporting files: backend/app/models/student_profile.py; backend/app/schemas/student_profile.py; frontend/src/pages/StudentDashboard.jsx; frontend/src/services/studentService.js. Suggested evidence: redacted profile form.
- Job search
  Implemented: students can browse jobs and search using text and city.
  How it works and why it matters: StudentJobs.jsx requests the job list through

jobService.js so students can narrow the available postings.
Supporting files: frontend/src/pages/StudentJobs.jsx; frontend/src/services/jobService.js; backend/app/api/routes/jobs.py; backend/tests/test_jobs.py. Suggested evidence: student job results with the search box and city filter.

- Job filters
  Implemented: job filtering supports search text, city, employment category, and virtual or in-person format.
  How it works and why it matters: the frontend maps Internship, Full-time, Part-time, and On-campus when supported to the backend's canonical employment-category values.
  Supporting files: frontend/src/pages/StudentJobs.jsx; frontend/src/services/jobService.js; backend/app/models/job.py; backend/app/schemas/job.py; backend/tests/test_jobs.py. Suggested evidence: one result set for each employment category.
- Job details
  Implemented: students can open one job and view its details before saving or applying.
  How it works and why it matters: the job service requests a job by ID and the backend returns the matching job record.
  Supporting files: backend/app/api/routes/jobs.py; backend/app/models/job.py; backend/app/schemas/job.py; frontend/src/pages/StudentJobs.jsx; frontend/src/services/jobService.js. Suggested evidence: redacted job-details view.
- Resume upload
  Implemented: applications accept PDF resumes and reject unsupported file types or oversized files.
  How it works and why it matters: the application endpoint validates the upload and stores it outside tracked source files, keeping private resume contents out of Git.
  Supporting files: backend/app/api/routes/applications.py; backend/app/models/application.py; backend/app/schemas/application.py; backend/tests/test_applications.py; .gitignore. Suggested evidence: PDF submission success and invalid-file test results without showing the resume.
- Job applications
  Implemented: students can apply to jobs and view their own applications.
  How it works and why it matters: the backend associates each application with the selected job and authenticated student and protects the student's application list.
  Supporting files: backend/app/api/routes/applications.py; backend/app/models/application.py; backend/app/schemas/application.py; frontend/src/pages/StudentJobs.jsx; frontend/src/pages/MyApplications.jsx; backend/tests/test_applications.py. Suggested evidence: sanitized application success and application list.
- Application-status page
  Implemented: MyApplications.jsx shows the student's submitted jobs and current application status.
  How it works and why it matters: the page requests only the current student's applications, giving the student one place to check progress.
  Supporting files: frontend/src/pages/MyApplications.jsx;

frontend/src/services/jobService.js; backend/app/api/routes/applications.py; backend/app/schemas/application.py. Suggested evidence: redacted status page.

Your partner

Build the company workflow:

- Company profile
- Create job postings
- View owned jobs
- View applicants
- View student profiles
- Update application status
- Company dashboard

Both partners

- Connect frontend pages to backend APIs.
- Add input validation.
- Add authorization checks.
- Test the complete job workflow.

End-of-week goal

This complete process should work:

Company creates job
↓
Student searches for job
↓
Student applies with resume
↓
Company views application
↓
Company changes application status
↓
Student sees the updated status

Week 3 — Events, data, and AI assistant

You

Build:

- Student event search
  Implemented: students can browse available events in the React event page.
  How it works and why it matters: eventService.js requests event records from the FastAPI event routes, including the Pair 16 cities Sunnyvale, Cupertino, and Mountain View.
  Supporting files: frontend/src/pages/StudentEvents.jsx; frontend/src/services/eventService.js; backend/app/api/routes/events.py; backend/app/models/event.py; backend/tests/test_events.py. Suggested evidence: student events page.
- Event filtering
  Implemented: event search supports text, city, event type, virtual or in-person format, and date range.
  How it works and why it matters: the frontend sends selected filters and the backend applies them before returning matching events.
  Supporting files: frontend/src/pages/StudentEvents.jsx; frontend/src/services/eventService.js; backend/app/api/routes/events.py; backend/app/schemas/event.py; backend/tests/test_events.py. Suggested evidence: filtered event results.
- Event details
  Implemented: students can open an event and review its details before registering.
  How it works and why it matters: the details route returns the selected event and the page presents its description, time, location or format, and registration control.
  Supporting files: backend/app/api/routes/events.py; backend/app/models/event.py; backend/app/schemas/event.py; frontend/src/pages/StudentEvents.jsx. Suggested evidence: event-details view.
- Event registration
  Implemented: authenticated students can register for events, and duplicate registration is prevented.
  How it works and why it matters: the backend checks the student-event pair before creating a registration, keeping registration records accurate.
  Supporting files: backend/app/api/routes/events.py; backend/app/models/event_registration.py; backend/app/models/event.py; frontend/src/services/eventService.js; backend/tests/test_events.py. Suggested evidence: successful registration and duplicate-registration result.
- Registered-events page
  Implemented: students can view events for which they are registered.
  How it works and why it matters: StudentRegisteredEvents.jsx requests registrations for the authenticated student and displays the related event information.
  Supporting files: frontend/src/pages/StudentRegisteredEvents.jsx; frontend/src/services/eventService.js; backend/app/api/routes/events.py; backend/app/models/event_registration.py. Suggested evidence: registered-events page with personal details redacted.
- Ollama client class
  Implemented: OllamaClient provides complete(messages, tools) and keeps chat()

compatibility.
How it works and why it matters: the assistant loop sends messages and tool definitions through one client class, including qwen3:8b in the verified live experiment.
Supporting files: backend/app/services/ollama_client.py; backend/app/assistant/loop.py; backend/tests/test_assistant_loop.py. Suggested evidence: sanitized client-interface test.

- Basic assistant conversation loop
  Implemented: the assistant accepts a student message, may call a hand-built tool, receives the result, and returns a final answer.
  How it works and why it matters: the loop identifies the student, persists conversations and tool logs, and stops after five iterations. The verified seed counts were 200 students, 50 companies, 300 jobs, 1,500 applications, 100 events, and 1,482 event registrations.
  Supporting files: backend/app/api/routes/assistant.py; backend/app/assistant/loop.py; backend/app/assistant/tools.py; backend/app/models/assistant_conversation.py; backend/app/models/assistant_tool_log.py; backend/tests/test_assistant_api.py; backend/tests/test_assistant_loop.py. Suggested evidence: sanitized assistant chat and persistence counts.

Your partner

Build:

- Company event creation
- View registered students
- Event eligibility checks
- Seed-data generator
- AI assistant tools:
  - `get_student_preferences`
  - `search_jobs`
  - `get_job_details`
  - `search_events`
  - `save_job_or_event`

Both partners

- Generate the required data:
  - 300 jobs
  - 1,500 applications
  - 200 students
  - 50 companies
  - 100 events
- Add the assistant chat interface.
- Add tool schemas.
- Add tool-call logging.

- Add conversation history.
- Add the iteration ceiling.
- Test the five required assistant scenarios.

End-of-week goal

The full application should work, and the assistant should be able to search jobs and events using the required tools.

Week 4 — Safety, testing, report, and submission

You

Focus on:

- Confirmation gate
  Implemented: read-only assistant tools can continue, while save actions are checked by a separate confirmation gate.
  How it works and why it matters: the gate makes a student's approval decision explicit before the assistant changes saved data.
  Supporting files: backend/app/assistant/confirmation.py; backend/app/assistant/loop.py; backend/tests/test_confirmation.py. Suggested evidence: focused approval and refusal tests.
- `confirm_action(tool_name, arguments, auto_approve=False)`
  `Implemented: the separate confirmation module defines confirm_action(tool_name, arguments, auto_approve=False).`
  `How it works and why it matters: read-only tools do not require confirmation, while save_job_or_event requires explicit student confirmation unless a controlled test enables auto approval.`
  `Supporting files: backend/app/assistant/confirmation.py; backend/app/assistant/tools.py; backend/tests/test_confirmation.py. Suggested evidence: sanitized confirmation-gate test output.`
- Save cancellation behavior
  Implemented: a declined save returns a cancellation string, the assistant acknowledges the refusal, and the save is not retried automatically.
  How it works and why it matters: the cancellation is recorded by the assistant loop and no saved item is created.
  Supporting files: backend/app/assistant/confirmation.py; backend/app/assistant/loop.py;

backend/app/assistant/tools.py; backend/tests/test_confirmation.py. Suggested evidence: refusal transcript and zero-save result.

- Prompt-injection defense
  Implemented: retrieved job and event descriptions are treated as untrusted data.
  How it works and why it matters: the prompt delimits retrieved content, labels it as untrusted, and separates it from system instructions so a poisoned description cannot control tool selection or request private student data.
  Supporting files: backend/app/assistant/loop.py; backend/app/assistant/tools.py; backend/tests/test_prompt_injection.py; docs/prompt_injection_before_after.md. Suggested evidence: redacted poisoned-content test.
- Before-and-after injection tests
  Implemented: the same five injection verification queries were run before and after the defense.
  How it works and why it matters: the recorded comparison measures the safety change rather than only describing it. Results changed from 4 of 5 unsafe before the defense to 0 of 5 unsafe after the defense.
  Supporting files: docs/prompt_injection_before_after.md; backend/tests/test_prompt_injection.py; backend/app/assistant/loop.py. Suggested evidence: before-and-after results table.
- Assistant transcripts
  Implemented: sanitized assistant transcripts record the user request, every tool call, arguments, result, final answer, and tool-call count.
  How it works and why it matters: the evidence omits passwords, tokens, private emails, and private student information. The deterministic reliability experiment used 20 turns with one bounded repair path. Separately, the live Ollama experiment used 26 turns on isolated database p16_handshake_reliability through backend port 9161 with qwen3:8b and no repair attempts; it recorded 15 valid/correct turns, 5 Ollama errors, 3 wrong tool results, 2 lost multi-step chains, 1 API/database error, 24 tool calls, 4 blocked save attempts, 0 approved saves, and 0 unintended saved items.
  Supporting files: docs/evidence/live_reliability_2026-10-19/live_transcripts.md; docs/evidence/live_reliability_2026-10-19/live_reliability_results.md; backend/app/assistant/tools.py; backend/app/assistant/loop.py. Suggested evidence: one sanitized transcript showing the full tool-call path.

Your partner

Focus on:

- Reliability testing across at least 20 assistant turns
- Failure classification
- Repair and reprompt path
- API testing
- `METRICS.md`
- `RUN_LOG.txt`

- Screenshots and evidence

Both partners

Complete:

- `README.md`
- `AI_USE.md`
- API documentation
- Five required transcripts
- Reliability table
- Prompt-injection results
- Confirmation-gate evidence
- Architecture diagram
- Database screenshots
- Five-minute recording
- Git history screenshot
- Final PDF report
- Final system testing
- `lab1` Git tag

End-of-week goal

The repository, report, recording, and application should all be ready for submission.

Suggested weekly Git milestones

Time	Milestone
End of Week 1	Authentication, database, and basic frontend
End of Week 2	Student/company job workflow
End of Week 3	Events, seed data, and AI assistant
End of Week 4	Safety testing, report, recording, and final submission

The most important rule is to finish a basic working version of every required feature before adding advanced styling or extra functionality.